

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Experiment	Experiment 5 – CNN Training and Optimization using MobileNetV2	

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, hyperparameter tuning, transfer learning, fine-tuning, and cross-validation on image classification performance.

The Oxford-IIIT Pet dataset was used with a MobileNetV2 architecture pretrained on ImageNet. Different experimental configurations were evaluated and the best configuration was selected using 5-fold cross-validation. Finally, the selected model was retrained on the complete training set and evaluated on an independent test set.

2. Experimental Environment and Configuration

The experiment was executed using TensorFlow and TensorFlow Datasets in Google Colab with GPU acceleration.

Parameter	Value
Framework	TensorFlow 2.20.0
Dataset	Oxford-IIIT Pet Dataset
Number of Classes	37
Image Size	$224 \times 224 \times 3$
Training Images	3680
Independent Test Images	3669
Training Split	2944 images
Validation Split	736 images
Default Batch Size	32
Short Experimental Epochs	3
Cross-Validation Epochs	2
Final Training Epochs	8
Random Seed	42
Pretrained Model	MobileNetV2 with ImageNet weights

The data for training was divided into 80% training data and 20% validation data through stratified sampling. Independent test data was not used at all in the process of selecting hyperparameters.

Preprocessing of input images was done by applying the preprocessing function of MobileNetV2. Data Augmentation included random horizontal flip, rotation, and zooming.

3. Dataset Visualization

The Oxford-IIIT Pet dataset contains images from 37 different breeds of cats and dogs. The images have different original dimensions and were resized to $224 \times 224 \times 3$ before model training.

Oxford-IIIT Pet Dataset Samples



Figure 1: Sample images from the Oxford-IIIT Pet dataset.

Inference: The dataset exhibits a lot of variation in animal pose, background, illumination, color, and viewpoint. This makes classification challenging and justifies the need for ImageNet-pretrained feature extractors and data augmentation.

4. MobileNetV2 Architecture

MobileNetV2 is a computationally efficient convolutional neural network designed for mobile and resource-constrained environments. Its major components include depthwise separable convolutions, pointwise convolutions, inverted residual blocks, and linear bottlenecks.

A simplified architecture used in this experiment is shown below.



The baseline classifier consists of a Global Average Pooling layer followed by a Dense layer with 256 units and ReLU activation. Depending on the experiment, Batch Normalization, L2 regularization, and Dropout were added.

5. Baseline MobileNetV2 Model

The baseline model used the following configuration:

Base Model	MobileNetV2
Pretrained Weights	ImageNet
Base Model	Frozen
Dense Layer	256 units, ReLU
Dropout Rate	0.25
Learning Rate	0.001
Optimizer	Adam
Loss Function	Sparse Categorical Cross-Entropy
Training Epochs	3

The baseline model achieved the following best validation accuracy:

Best Baseline Validation Accuracy = 88.99%

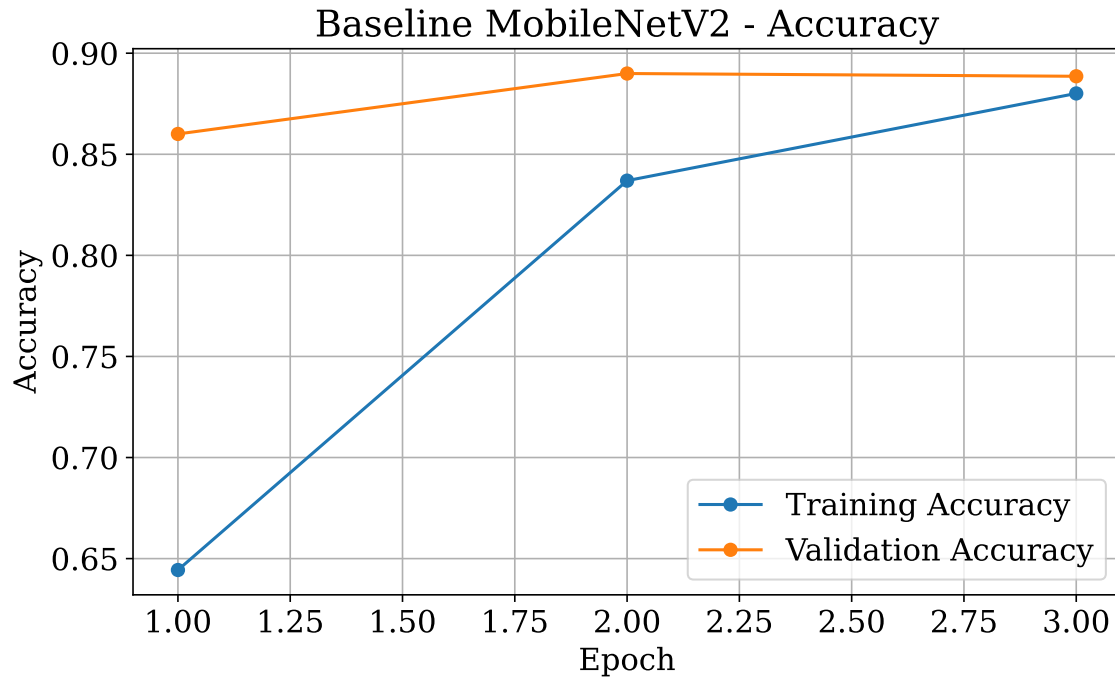


Figure 2: Training and validation accuracy of the baseline MobileNetV2 model.

Inference: Training accuracy improved from around 64.4% to 88.0% across three epochs, whereas validation accuracy achieved around 89.0%. The slight disparity between the two indicates that the pretrained MobileNetV2 learned features performed well for the particular dataset.

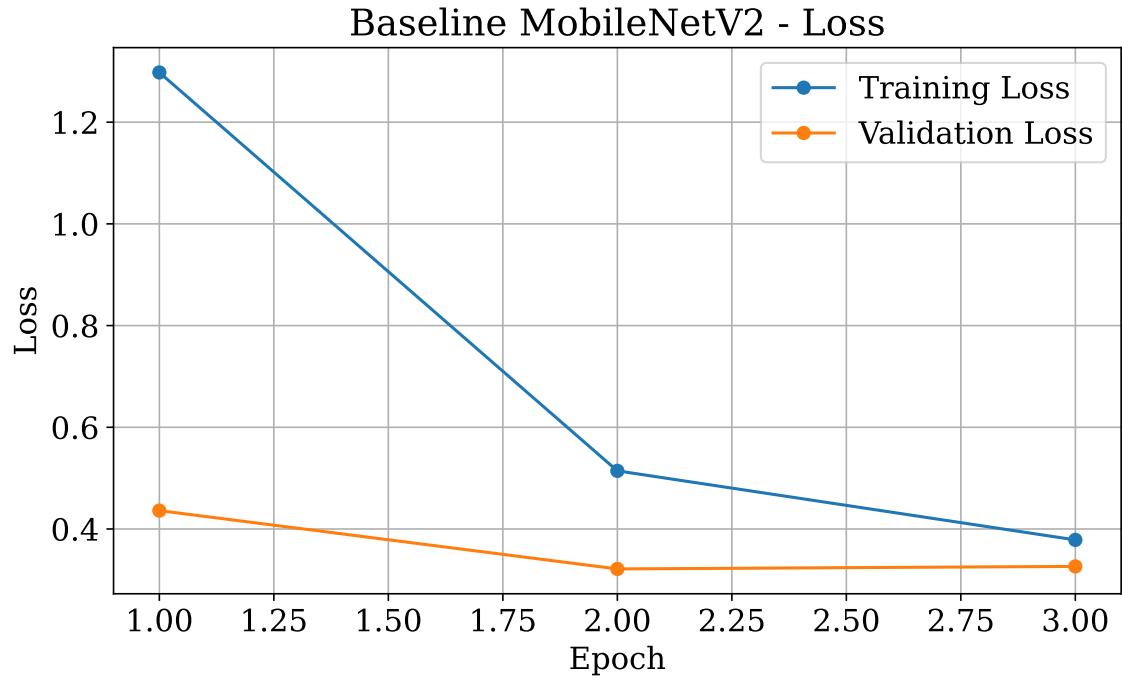


Figure 3: Training and validation loss of the baseline MobileNetV2 model.

Inference: Training as well as validation losses both reduced very fast during the process of training. Validation loss stayed quite low and steady without much indication of overfitting in the three epoch baseline study.

6. Weight Initialization Study

Four initialization strategies were compared for the newly added Dense layer:

- Zero initialization
- Random Normal initialization
- Xavier/Glorot initialization
- He Normal initialization

The experimental results are shown below.

Initialization Method	Best Validation Accuracy
Zeros	2.72%
Random Normal	88.72%
Xavier/Glorot	88.04%
He Normal	88.72%

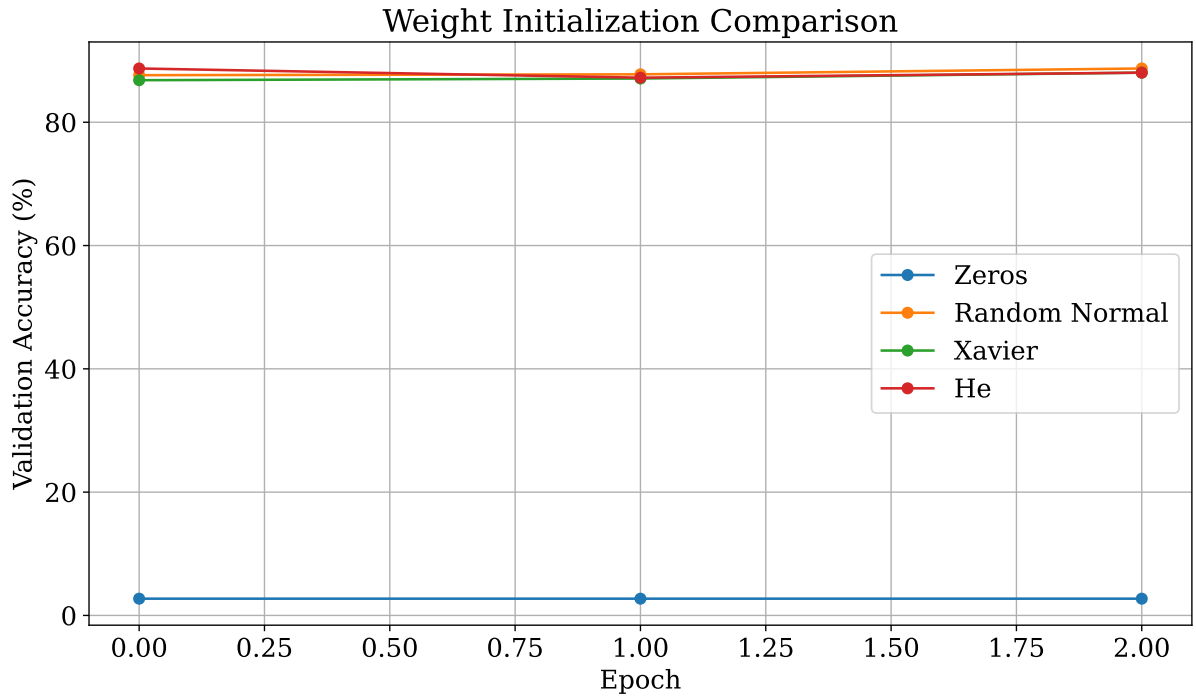


Figure 4: Comparison of validation accuracy for different weight initialization strategies.

Inference: Zero initialization failed miserably since all neurons had same starting values hence symmetric representations. Random Normal and He Initialization yielded the best validation accuracy while Xavier Initialization gave good results too.

7. Regularization and Batch Normalization

Four regularization strategies were compared:

- No regularization
- L2 regularization with $\lambda = 0.001$
- Dropout with rate 0.5
- Batch Normalization

Method	Best Validation Accuracy
No Regularization	90.08%
L2 Regularization	88.04%
Dropout	89.81%
Batch Normalization	89.67%

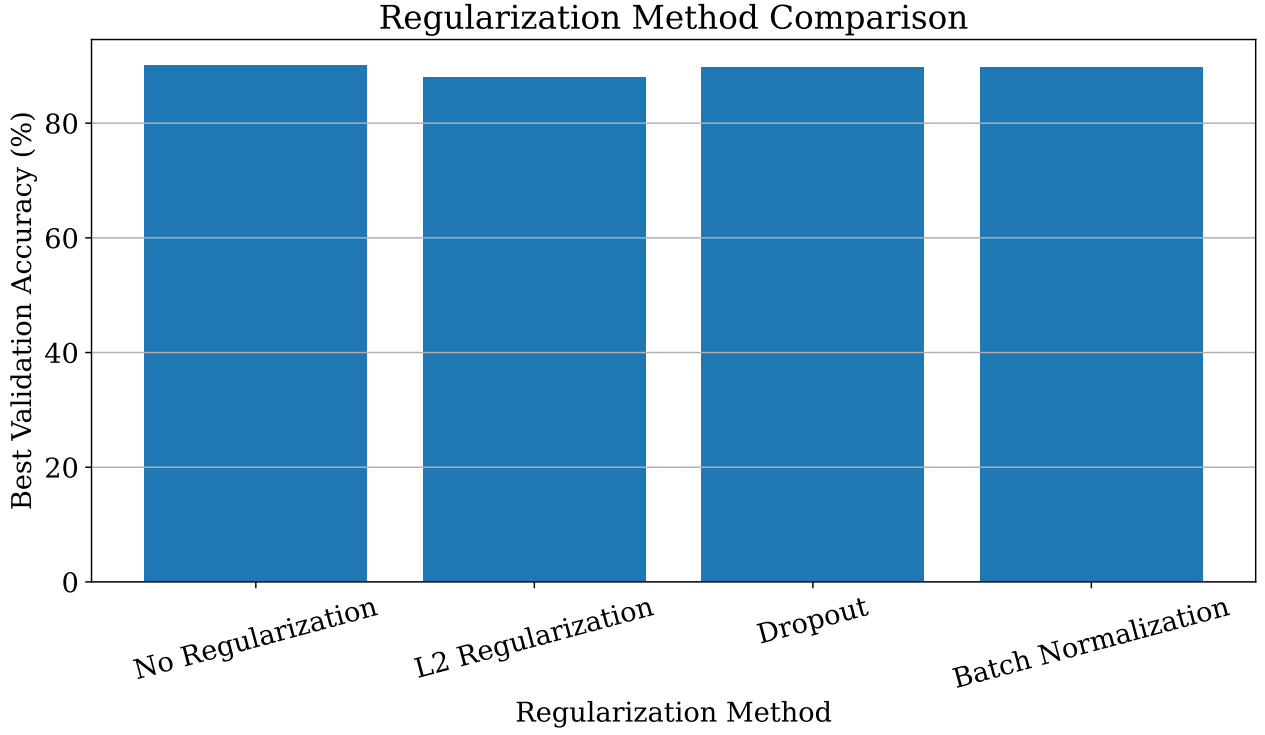


Figure 5: Comparison of different regularization methods.

Inference: Validation results were quite satisfactory for all of them. The best result for such a small experiment was acquired using no further regularization with 90.08%, and Dropout and Batch Normalization were quite close to each other. L2 regularization had the worst result out of the four applied regularization approaches.

8. Batch Normalization

Batch Normalization normalizes activations within a mini-batch. For a batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable scale and shift parameters.

In the experimental comparison, Batch Normalization achieved a best validation accuracy of 89.67%, which was close to the other high-performing regularization configurations.

9. Optimizer Comparison

The following optimization algorithms were compared:

- Stochastic Gradient Descent (SGD)
- SGD with Momentum
- RMSProp
- Adam

Optimizer	Final Loss	Best Val. Accuracy	Best Epoch	Training Time (s)
SGD	3.4435	18.89%	3	33.75
Momentum	1.2719	83.70%	3	33.93
RMSProp	0.3921	88.72%	3	33.18
Adam	0.3787	88.99%	2	33.13

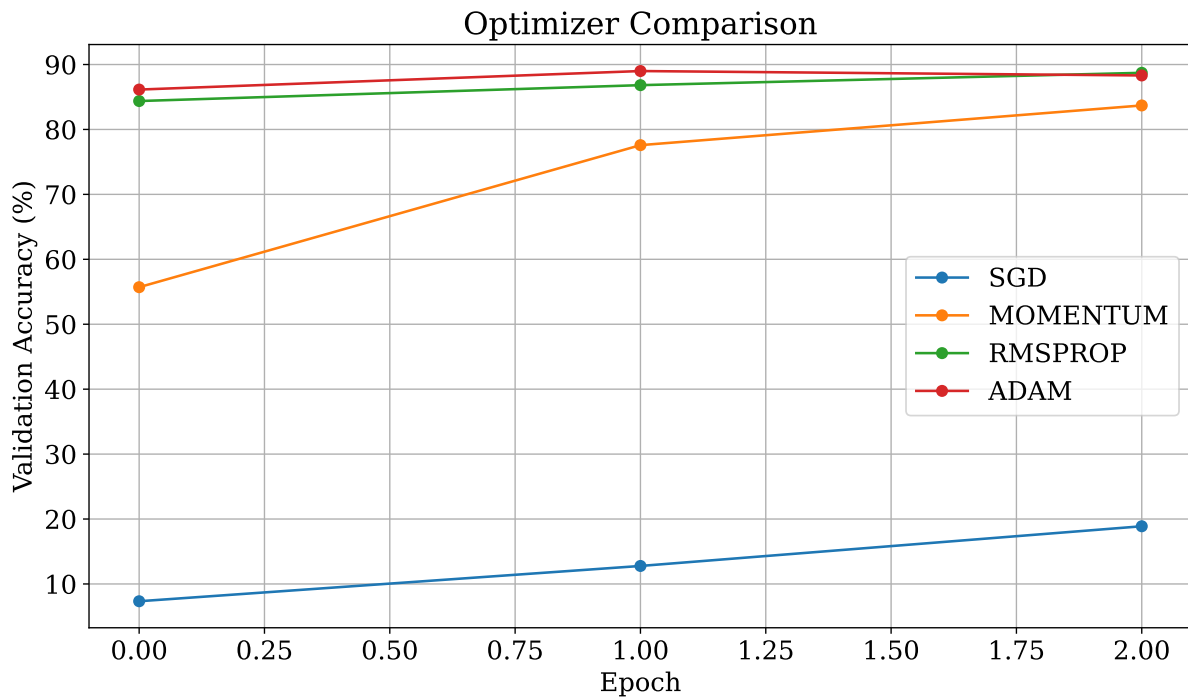


Figure 6: Validation accuracy comparison of different optimizers.

Inference: The SGD took time to converge and resulted in only 18.89% validation accuracy. The momentum greatly helped to converge, whereas RMSProp and Adam performed the best. The Adam had the highest validation accuracy of 88.99% and attained its best result in epoch 2.

10. Learning Rate Tuning

Two learning rates were evaluated while keeping the remaining configuration fixed.

Learning Rate	Best Validation Accuracy
0.001	86.82%
0.0001	83.83%

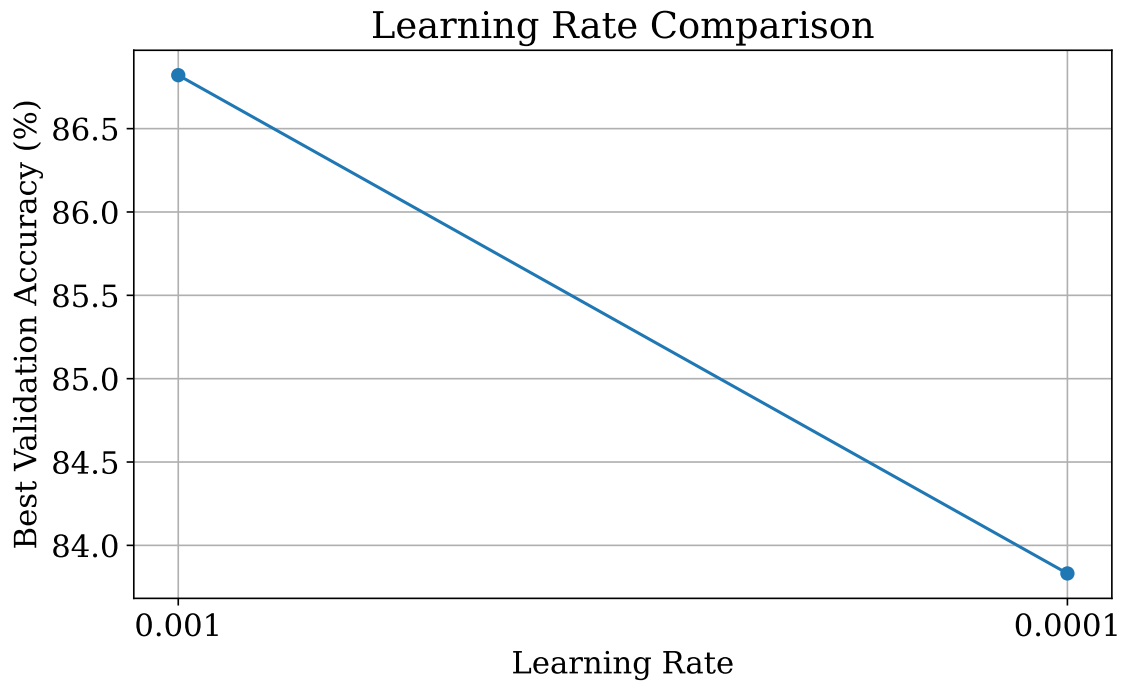


Figure 7: Effect of learning rate on best validation accuracy.

Inference: Learning rate 0.001 performed better validation accuracy compared to 0.0001 in terms of time available for training. The low learning rate took more time to converge and showed that a very small learning rate might take more epochs.

11. Batch Size Tuning

Batch sizes of 16, 32, and 64 were compared.

Batch Size	Best Validation Accuracy
16	87.91%
32	84.51%
64	78.40%

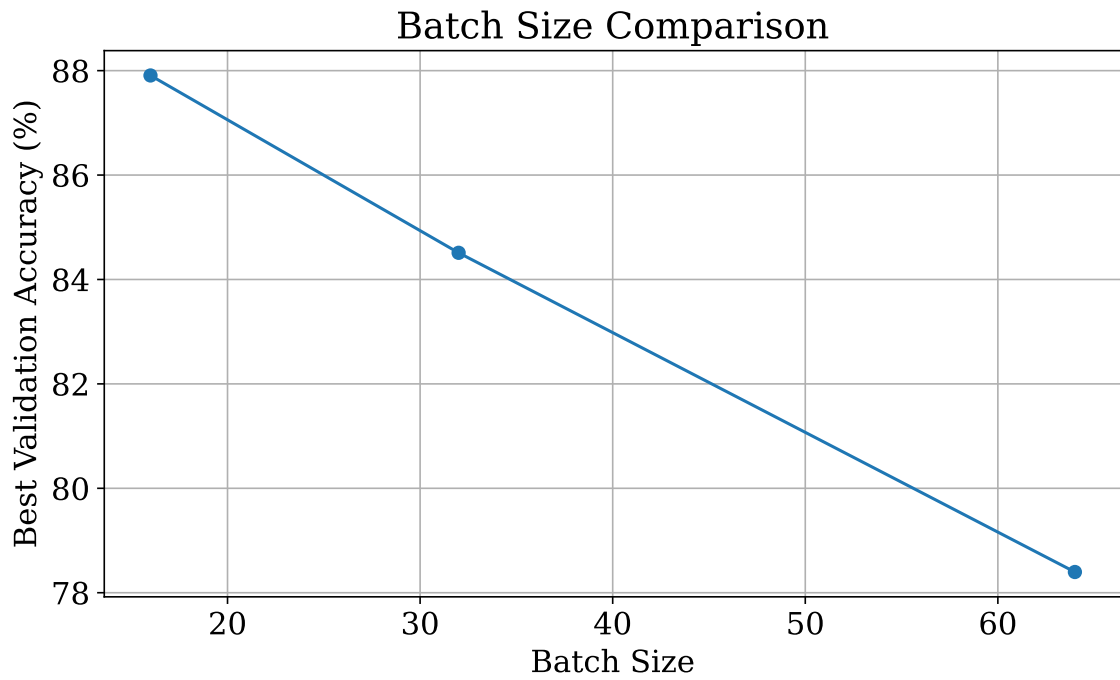


Figure 8: Effect of batch size on best validation accuracy.

Inference: The batch size of 16 provided the best accuracy on the validation set. As the batch size was increased from 16 to 64, the performance was decreased. This is probably because of the more frequent updating of the parameters.

12. Dropout Rate Tuning

Three Dropout rates were evaluated.

Dropout Rate	Best Validation Accuracy
0.00	85.19%
0.25	85.87%
0.50	83.02%

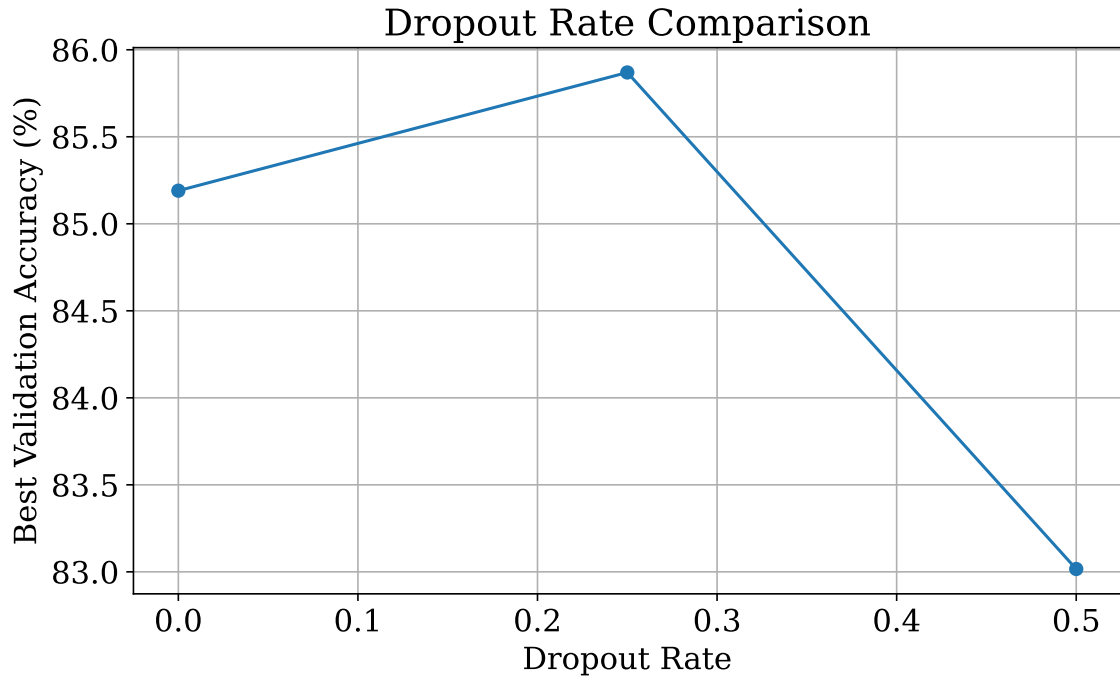


Figure 9: Effect of dropout rate on validation accuracy.

Inference: Dropout of 0.25 gave the highest validation accuracy. Zero Dropout gave a little lower accuracy, whereas the Dropout of 0.5 gave lower accuracy, implying that too much regularization hindered the learning capability of the model.

13. Transfer Learning: Feature Extraction

In the feature extraction experiment, the MobileNetV2 convolutional base was frozen. Only the newly added classification layers were trained. The model used Batch Normalization, a Dense layer with 256 ReLU units, and a Dropout rate of 0.25.

Training was performed for up to 8 epochs using early stopping. Training stopped after four epochs.

Best Feature Extraction Validation Accuracy = 88.59%

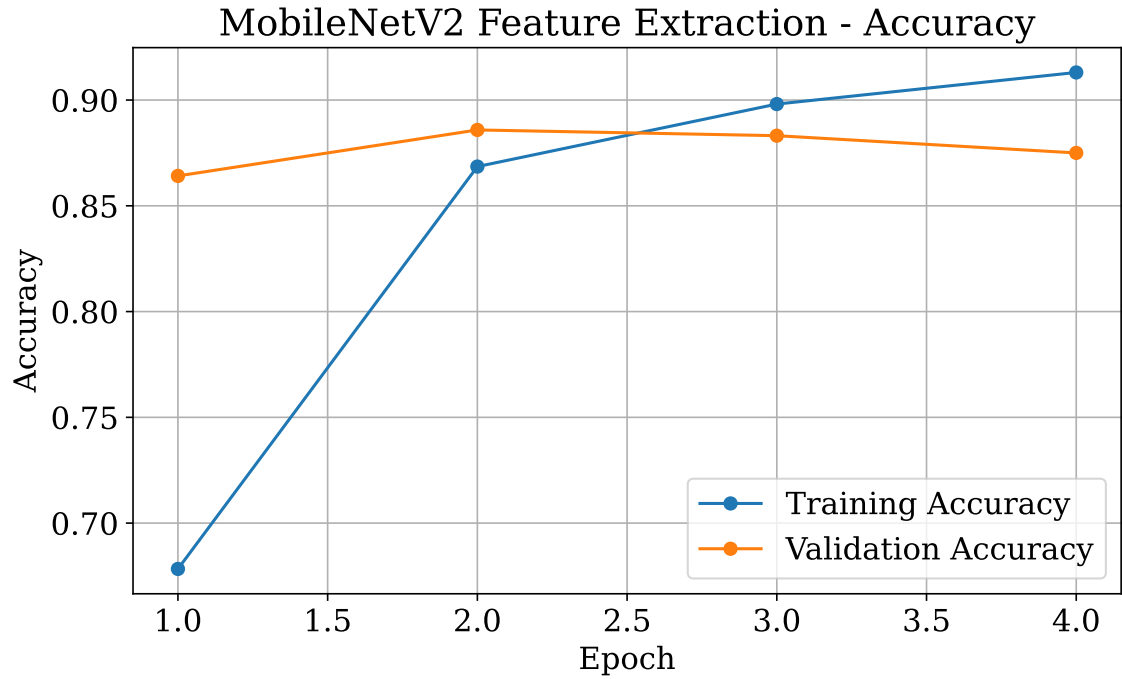


Figure 10: Training and validation accuracy during MobileNetV2 feature extraction.

Inference: The training accuracy increased gradually to about 91.3%, whereas the validation accuracy reached its peak value of around 88.6% in epoch 2 and after that started to decrease slightly. This is an indication that model fitting began improving post-best validation epoch.

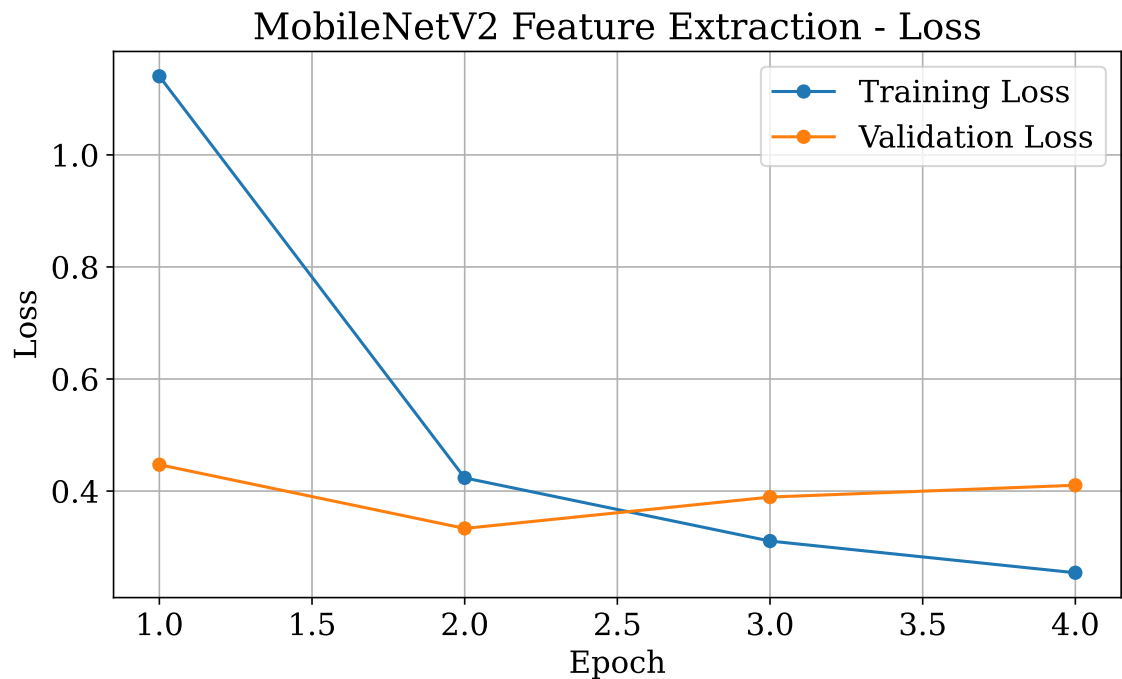


Figure 11: Training and validation loss during MobileNetV2 feature extraction.

Inference: Training loss reduced consistently from about 1.14 to 0.25. Validation loss attained its lowest value at epoch 2 after which it started increasing, thereby accounting for why early stopping helped in recovering the best weights.

14. Transfer Learning: Fine-Tuning

For fine-tuning, the MobileNetV2 base was made trainable, while the first 100 layers were frozen. The upper layers were fine-tuned using Adam with a smaller learning rate of

10^{-5} .

The fine-tuning configuration was:

Base Model	MobileNetV2
Frozen Layers	First 100 layers
Remaining Layers	Trainable
Batch Normalization	Enabled
Dense Layer	256 units with ReLU
Dropout	0.25
Optimizer	Adam
Learning Rate	10^{-5}
Maximum Epochs	8

The best validation accuracy was

Best Fine-Tuning Validation Accuracy = 77.99%

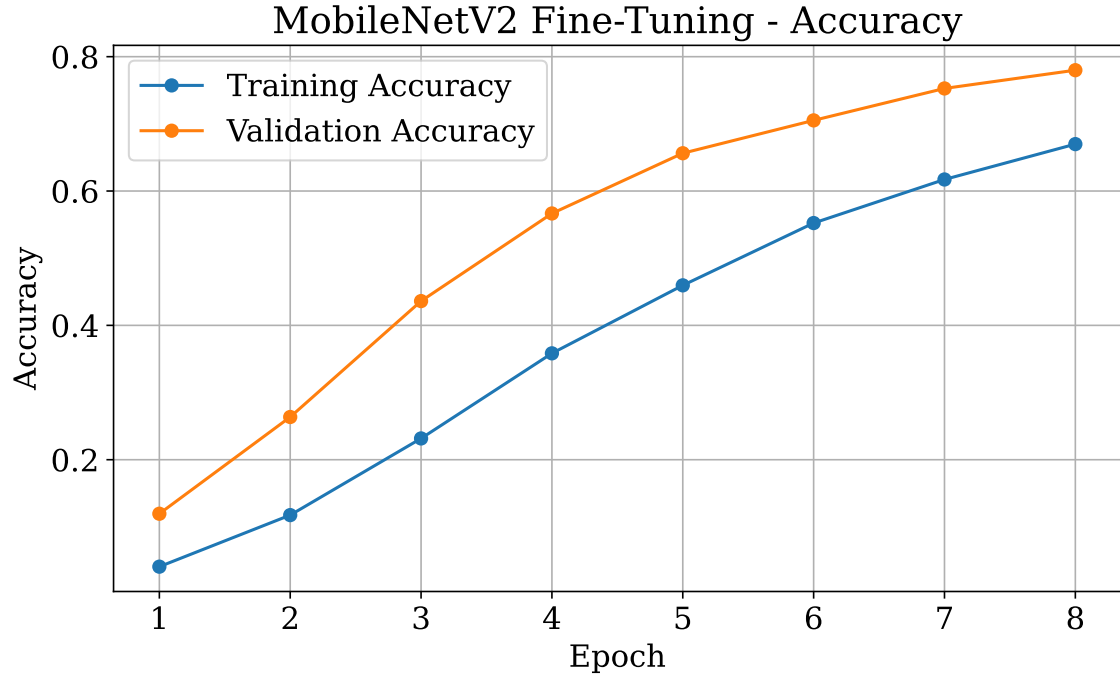


Figure 12: Training and validation accuracy during MobileNetV2 fine-tuning.

Inference: Both training and validation accuracies kept increasing consistently during the eight epochs. Nevertheless, the resulting validation accuracy, which was around 78.0%, turned out to be lower than the one achieved during the process of feature extraction.

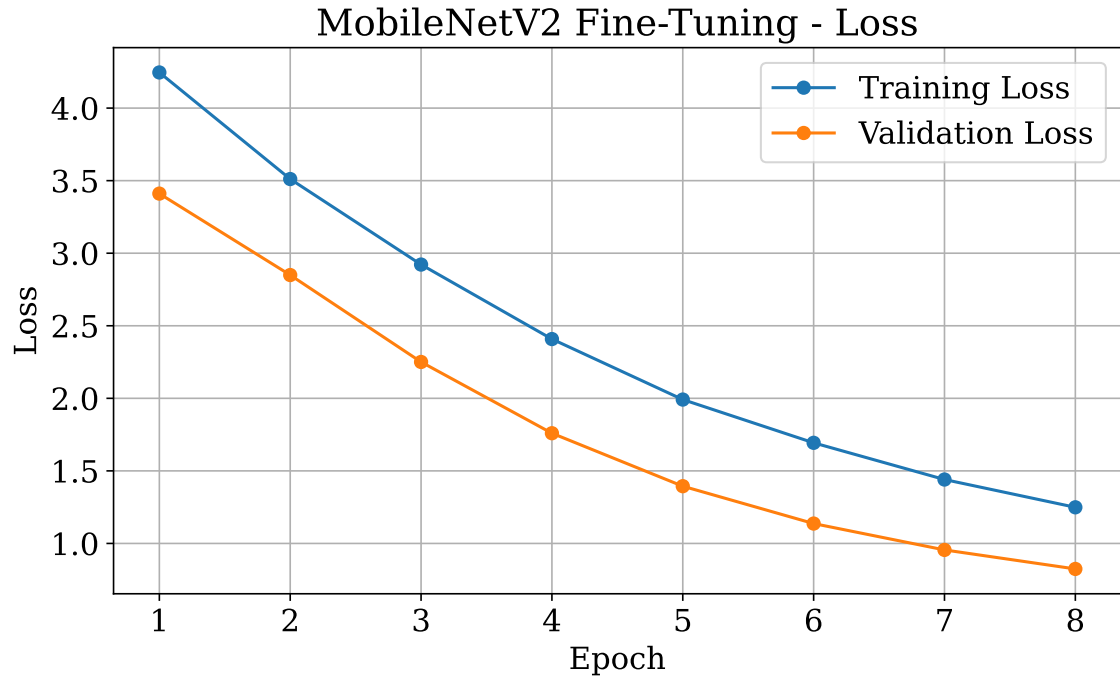


Figure 13: Training and validation loss during MobileNetV2 fine-tuning.

Inference: The losses were consistently falling from around 4.25 and 3.41 to about 1.25 and 0.82. This model showed improvement until epoch 8 and hence there is a potential to make it better through further fine-tuning.

15. 5-Fold Cross-Validation

Three promising configurations were evaluated using stratified 5-fold cross-validation on the complete original training dataset.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1 Baseline	88.72%	91.03%	89.13%	88.18%	88.32%	89.08% $\pm$ 1.03%
C2 BatchNorm	81.52%	80.03%	79.89%	77.99%	79.48%	79.78% $\pm$ 1.13%
C3 L2 + Dropout	73.51%	74.59%	75.95%	72.96%	77.04%	74.81% $\pm$ 1.51%

The configurations were defined as follows:

Configuration	Dropout	L2	BatchNorm	Learning Rate	Optimizer
C1 Baseline	0.25	0.0	No	0.001	Adam
C2 BatchNorm	0.25	0.0	Yes	0.0001	Adam
C3 L2 + Dropout	0.50	0.001	Yes	0.0001	Adam

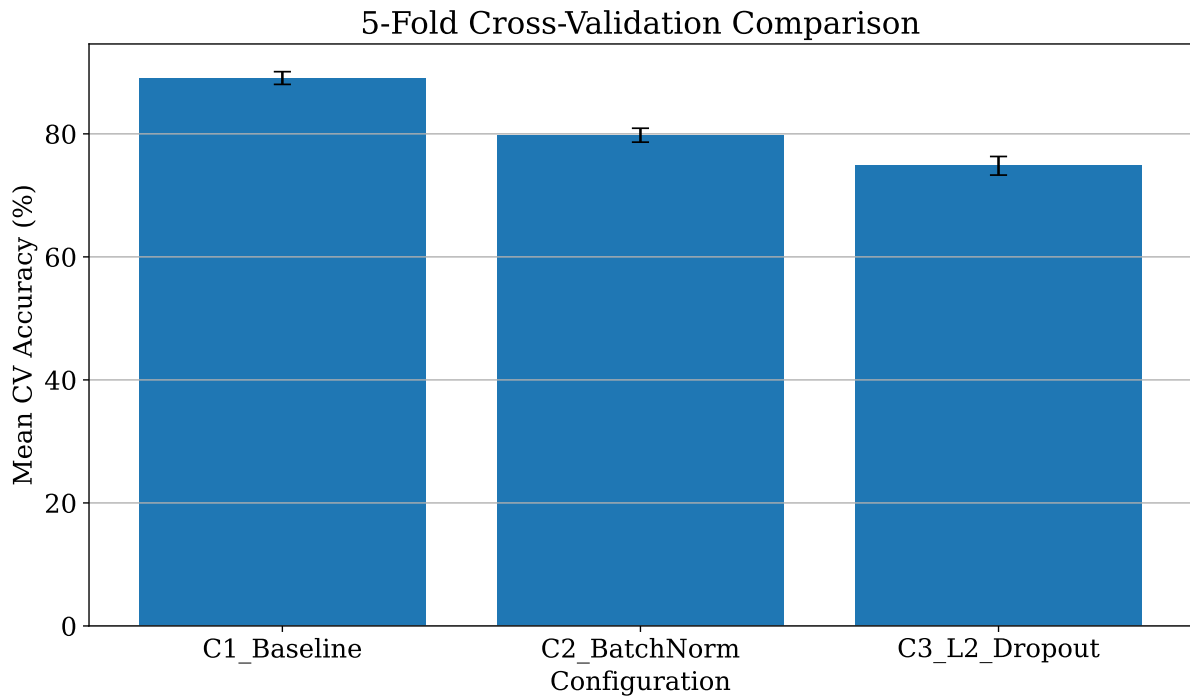


Figure 14: Mean 5-fold cross-validation accuracy with standard deviation error bars.

Inference: Configuration C1 Baseline reached the highest mean cross-validation accuracy of 89.08% and also demonstrated minimal variance with a standard deviation of about 1.03%. Hence, it was chosen as the optimal configuration for training and testing.

16. Best Configuration

Based on the 5-fold cross-validation results, the selected configuration was

C1_Baseline

with the following hyperparameters:

Dropout Rate	0.25
L2 Regularization	0.0
Batch Normalization	Disabled
Learning Rate	0.001
Optimizer	Adam
Mean CV Accuracy	89.08%
CV Standard Deviation	1.03%

17. Final Model Training

The selected C1 Baseline configuration was retrained using all 3680 images in the original training dataset. The model was trained for 8 epochs.

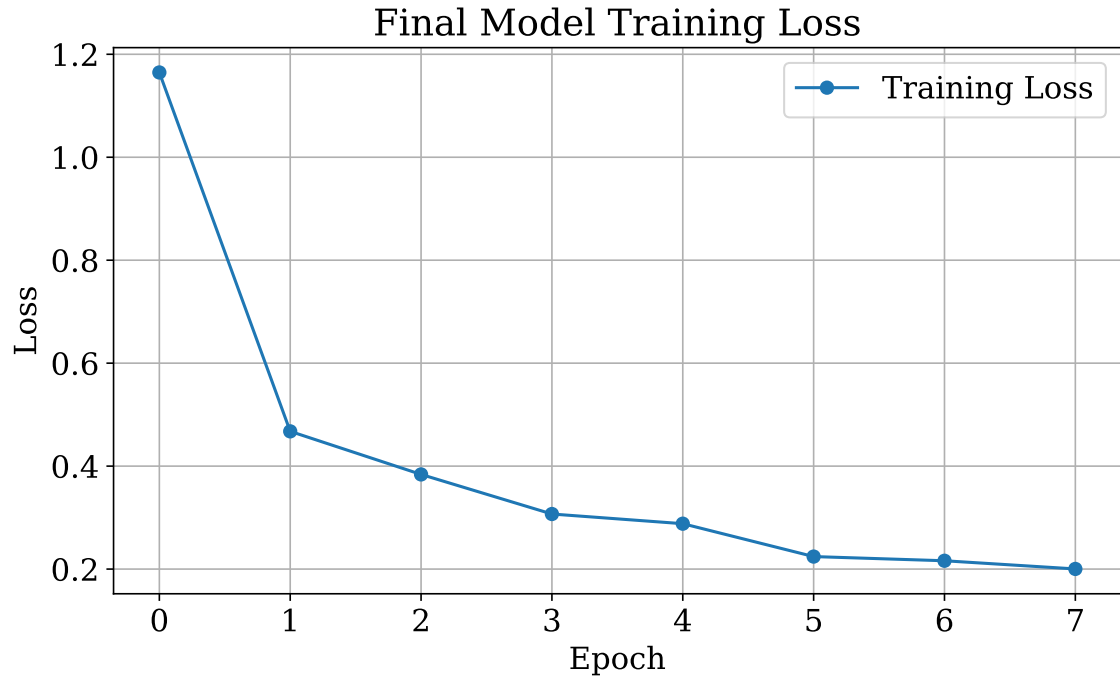


Figure 15: Training loss of the final selected model.

Inference: Training loss gradually dropped down from about 1.16 during the first epoch to 0.20 in the eighth epoch. This clearly shows that the optimization was done successfully and the final model converged successfully. The final training time was

72.88 seconds

18. Independent Test Set Evaluation

After selecting the best configuration through cross-validation, the final model was evaluated on the independent Oxford-IIIT Pet test set containing 3669 images.

The final evaluation metrics are shown below.

Metric	Value
Best Configuration	C1 Baseline
Mean 5-Fold CV Accuracy	89.08%
CV Standard Deviation	1.03%
Independent Test Accuracy	87.90%
Test Loss	0.3969
Weighted Precision	89.21%
Weighted Recall	87.90%
Weighted F1-Score	87.83%
Final Training Time	72.88 seconds
Total Parameters	2,595,429

The independent test accuracy is close to the mean cross-validation accuracy, which indicates that the selected model generalized reasonably well to unseen test images.

19. Confusion Matrix

The following confusion matrix summarizes the predictions of the final model over all 37 classes.

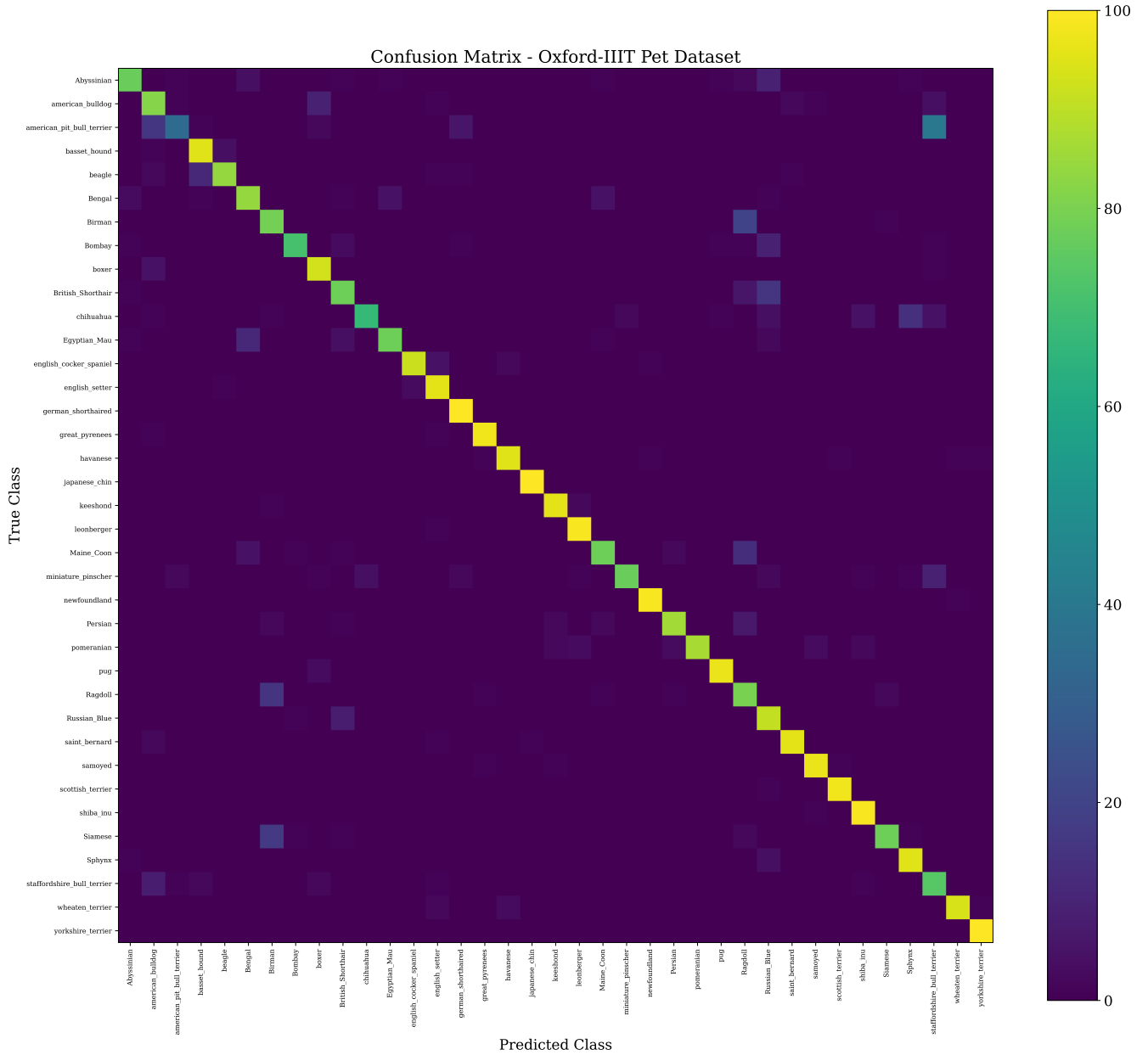


Figure 16: Confusion matrix for the final model on the independent test set.

Inference: Diagonal values for most classes are high, which means the correct classification of many samples. There are some similar breeds which demonstrate off-diagonal errors. This means that the pet breed classification task is difficult.

20. Class-wise Performance Analysis

The classification report showed strong performance for several classes. Examples include:

Class	Precision	Recall	F1-Score
japanese_chin	0.99	1.00	1.00
newfoundland	0.98	0.99	0.99
yorkshire_terrier	0.98	1.00	0.99
great_pyrenees	0.97	0.98	0.98
scottish_terrier	0.98	0.99	0.98
leonberger	0.94	0.99	0.97
pug	0.97	0.97	0.97

Some more challenging classes included:

Class	Precision	Recall	F1-Score
american_pit_bull_terrier	0.88	0.35	0.50
staffordshire_bull_terrier	0.55	0.83	0.66
Ragdoll	0.61	0.80	0.69
Birman	0.69	0.79	0.73
american_bulldog	0.69	0.82	0.75
Russian_Blue	0.66	0.91	0.76

The lower scores for some classes are consistent with visual similarities between related dog breeds and between cat breeds with similar coat patterns, facial structures, or body shapes.

21. Analysis of Misclassified Images

A total of

444

images were incorrectly classified out of 3669 independent test images.

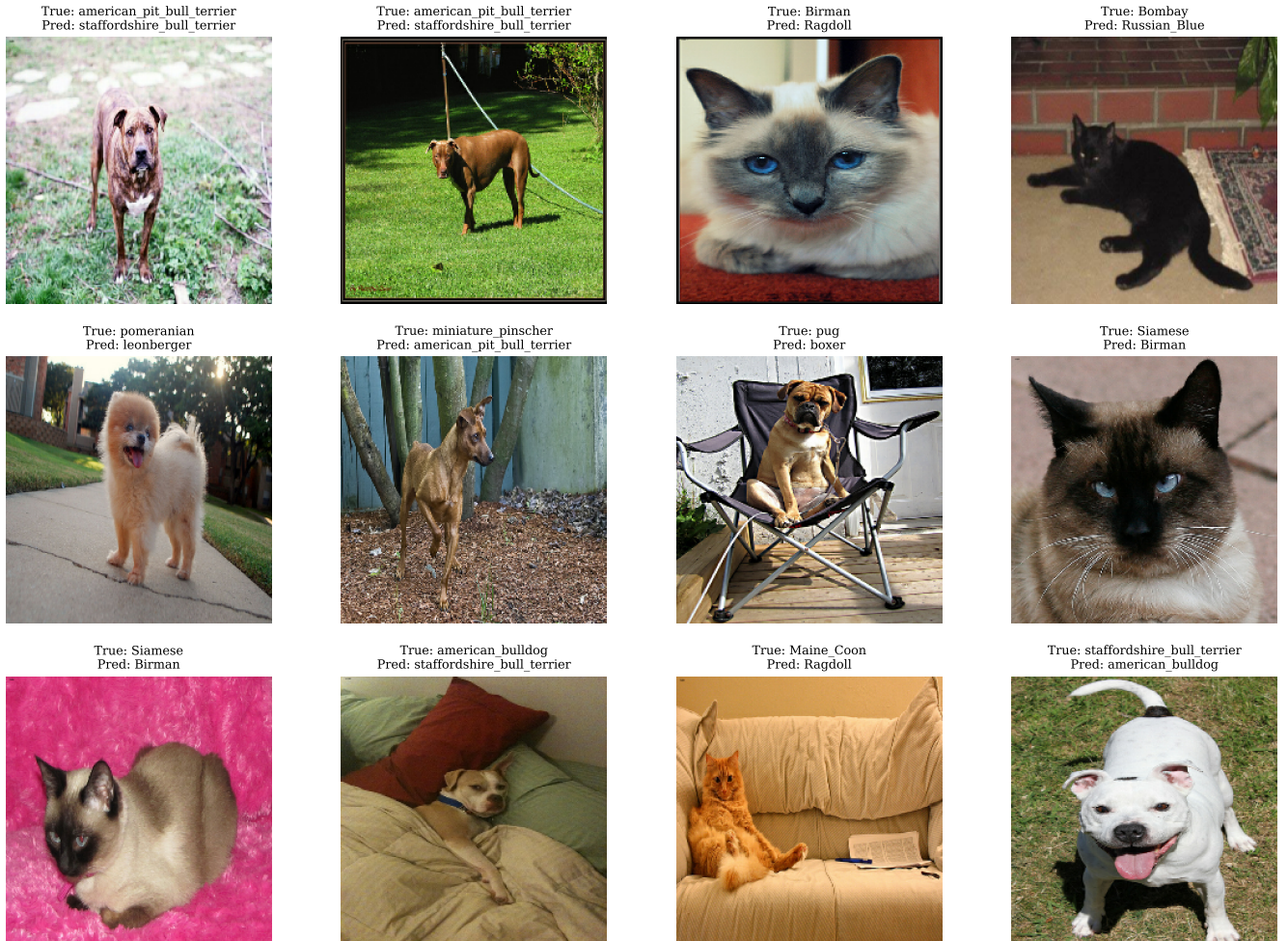


Figure 17: Representative examples of incorrectly classified test images.

Examples of observed misclassifications include:

- american_pit_bull_terrier → staffordshire_bull_terrier
- Birman → Ragdoll
- Bombay → Russian_Blue
- pomeranian → leonberger
- pug → boxer

- Siamese → Birman
- Maine_Coon → Ragdoll
- staffordshire_bull_terrier → american_bulldog

Inference: There are many mistakes made regarding breeds that look alike. The angle of the image, light, background, body position, coat, and partially seen animals also play a part in this issue.

22. Overall Experimental Results

Experiment	Best Result	Selected Option	Observation
Baseline Model	88.99%	MobileNetV2 + Adam	Strong baseline performance after only three epochs.
Weight Initialization	88.72%	Random Normal / He	Zero initialization failed because of symmetric learning.
Regularization	90.08%	No Regularization	Other methods were close, but L2 performed comparatively worse.
Optimizer	88.99%	Adam	Adam and RMSProp clearly outperformed SGD.
Learning Rate	86.82%	0.001	Higher than 0.0001 during the short training experiment.
Batch Size	87.91%	16	Smaller batch size performed best.
Dropout	85.87%	0.25	Moderate regularization was preferable to no or excessive Dropout.
Feature Extraction	88.59%	Frozen MobileNetV2	Strong performance with early stopping after four epochs.
Fine-Tuning	77.99%	Partial Unfreezing	Performance was still improving but did not surpass feature extraction.
Cross-Validation	89.08% $\pm$ 1.03%	C1 Baseline	Highest mean accuracy with low variability.
Final Test Performance	87.90%	C1 Baseline	Good generalization to the independent test set.

23. Discussion

1. Why is weight initialization important?

The Weight Initialization will decide where the optimization starts from. Poor weight initialization may result in slow convergence, bad gradients, or even symmetrical learning. Zero initialization performed very badly, while Random Normal, Xavier, and He initialization did far better in this case.

2. How can training and validation curves identify overfitting?

Overfitting is indicated if there is improvement in the training performance but validation performance remains unchanged or deteriorates. For the feature extraction task, the validation loss was minimum in epoch 2 but thereafter it increased whereas the training loss decreased.

3. How does Dropout affect the model?

During training, Dropout randomly turns off a percentage of neurons. The test proved that the dropout of 0.25 was more efficient than a dropout of 0.5, which means that an excessive Dropout may decrease the efficiency of the classifier.

4. Which optimizer performed best?

Adam had the highest accuracy of about 88.99%. RMSProp was very similar, while Momentum was moderately effective, and plain SGD took too long to converge.

5. Why is a smaller learning rate used for fine-tuning?

Useful image representation is already available in the pretrained weights. A big learning rate may change the learned features or ruin them. Thus, for fine-tuning, a smaller learning rate, which is 10^{-5} , was used.

6. Why is 5-fold cross-validation useful?

Cross-validation tests the configuration against various validation sets instead of doing this in one split alone. Both the mean accuracy and the standard deviation give us data regarding the performance and stability.

7. Why was C1 Baseline selected?

C1 Baseline obtained the highest cross-validation accuracy of 89.08% with a lower standard deviation of 1.03%. This model exhibited the best combination of performance and consistency.

24. Final Conclusion

These experiments provided a thorough investigation of CNN training and optimization on the Oxford-IIIT Pet dataset using a MobileNetV2 architecture pretrained on ImageNet.

It was shown that model performance strongly depends on the weight initialization, regularization, optimization method, learning rate, batch size and Dropout rate. Zero initialization was not effective, while Random Normal and He initialization resulted in high performance. Adam and RMSProp optimization algorithms clearly outperformed simple SGD, and the learning rate of 0.001 performed better than 0.0001 in this case.

As for the regularization experiments, all models gave relatively high performance, while no additional regularization was the best in terms of the single best validation accuracy. Model with feature extraction had the best validation accuracy of 88.59%, while the chosen fine-tuning model reached 77.99% and was still converging at the end of training.

The final configuration of the model was chosen based on 5-fold cross-validation. C1 Baseline had the mean validation accuracy of

$$89.08\% \pm 1.03\%$$

and was chosen for the final training. The trained model had independent test accuracy of

$$87.90\%$$

with weighted precision of 89.21%, weighted recall of 87.90% and weighted F1-score of 87.83%.

To conclude, this experiment showed that transfer learning using a well-optimized MobileNetV2 architecture can give good generalization performance on the problem of fine-grained classification of 37 pet breeds.

25. References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
2. S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation.
6. Keras Documentation.